```
        throw new IllegalArgumentException("invalid token",
e);
    }
}
```

#A Parse, decrypt, and validate the JWT using the decryption key.
#B Extract the JWT ID and revoke it from the DatabaseTokenStore whitelist.

The final part of the solution is to check that the whitelist token hasn't been revoked when reading a JWT token. As before, parse and decrypt the JWT using the decryption key. Then extract the JWT ID and perform a lookup in the `DatabaseTokenStore`. If the entry exists in the database, then the token is still valid, and you can continue validating the other JWT claims as before. But if the database returns an empty result then the token has been revoked and so is invalid. Update the `read()` method in JwtTokenStore.java to implement this addition check, as shown in listing 6.17. If you moved some attributes into the database, then you could also copy them to the token result in this case.

Listing 6.17 Checking if a JWT has been revoked

```
    var jwt = EncryptedJWT.parse(tokenId);    #A
    var decryptor = new DirectDecrypter(encKey);    #A
    jwt.decrypt(decryptor);    #A

    var claims = jwt.getJWTClaimsSet();
    var jwtId = claims.getJWTID();    #B
    if (tokenWhitelist.read(request, jwtId).isEmpty()) {
#B
        return Optional.empty();    #C
```